

Data Science

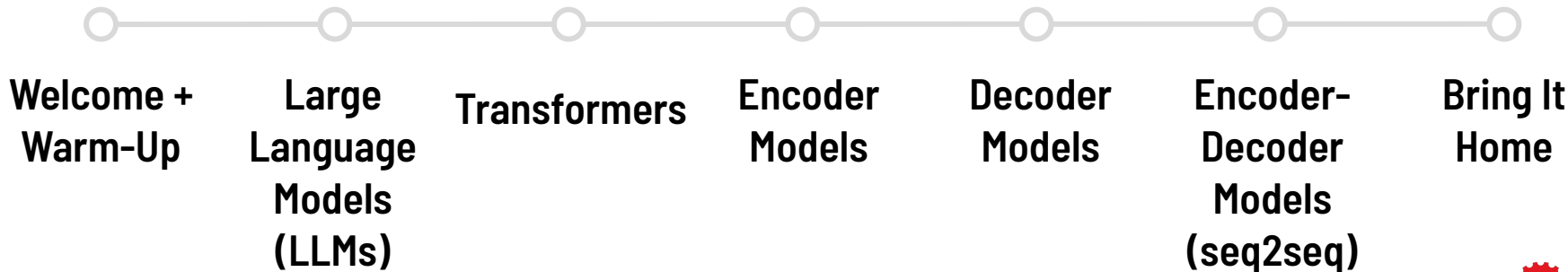
Introduction to Transformers

NLP and GenAI



GENERAL ASSEMBLY

Lesson roadmap



The background of the slide is a black and white photograph of a train station platform. Several people are walking away from the camera towards a bright light source at the end of the platform, creating long silhouettes. The floor is wet and reflective. On the left side of the image, there is a solid red vertical bar. Overlaid on this bar is a white rectangular box containing the text 'Warm-Up' in a large, bold, black sans-serif font.

Warm-Up

Transformer Architecture

Transformer architecture has enabled AI tools like ChatGPT, Google Search, and Google Translate. In this module you'll dive into how the transformer architecture works and apply models to solve a real-world problem.

- ☐ How do you think ChatGPT creates a response to your prompts?
- ☐ How does it break down your prompt?
- ☐ What other information does it use?
- ☐ How does the response get generated?

Learning objectives



1

Explain how probabilistic language models generate text.

2

Describe transformer architecture: encoders, decoders, and encoder-decoders.

3

Evaluate which architecture to use for common NLP business applications.

4

Apply encoder architecture to Named Entity Recognition (NER) task.

5

Apply decoder architecture to text generation task.

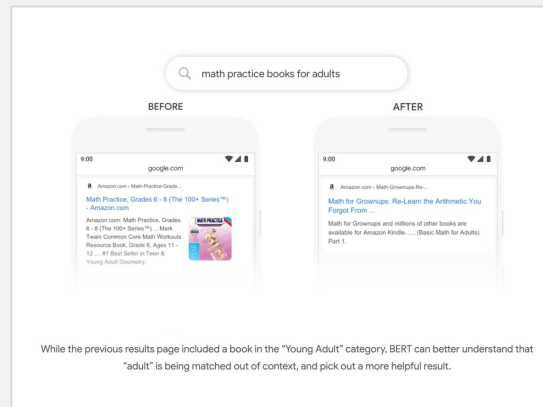
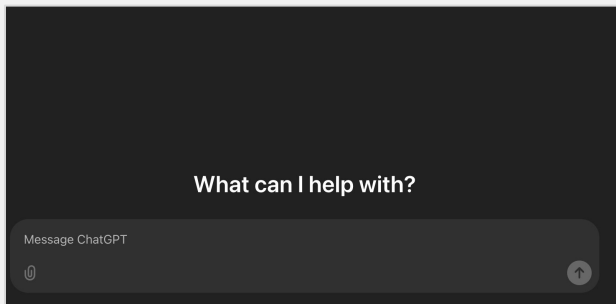
6

Apply encoder-decoder (seq2seq) architecture to translation and summarization task.

Large Language Models (LLMs)

AI powered by LLMs

There is a lot of excitement over AI. Let's peek under the hood and see how tools like ChatGPT and Google Translate work.



Language Models

A language model is a tool that uses probabilities to predict words in a sequence.

LLMs

are

????

Possible Word	Probability
fun	.30
neat	.25
interesting	.23
confusing	.22

Language Models

Typically it will randomly select a word among the top most likely options. Like this one: “fun”.

LLMs

are

????

Possible Word	Probability
fun	.30
neat	.25
interesting	.23
confusing	.22

Language Models

Or this one: “interesting”.

LLMs

are

????

Possible Word	Probability
fun	.30
neat	.25
interesting	.23
confusing	.22

LLMs

Consider this question:

Why wouldn't we select the word with the highest probability each time?

- ☐ What might happen if we did?
- ☐ What does the element of chance provide opportunity for?

Here's an example from where it got stuck in a loop

“The best JavaScript framework is the best JavaScript framework because the best JavaScript framework is the best JavaScript framework, so it is indeed the best JavaScript framework, and we must think about the best JavaScript framework, because after all, the best JavaScript framework is the best JavaScript framework...”

Large Language Models (LLMs)

- Trained on vast amounts of data.
- Predict the next word using probability.
- Powered by **transformers**.



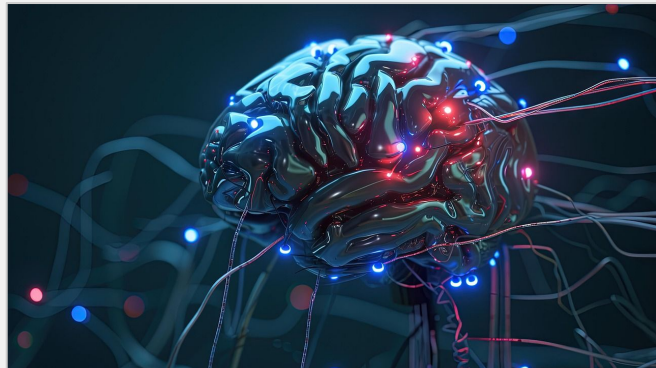
Transformers

Why Transformers?

Recurrent neural networks (RNNs) and Long short-term memory networks (LSTMs) - a more specialized type of RNN - are **neural networks** designed to **process sequential data**.

They maintain an internal memory of previous inputs, which allows them to capture temporal dependencies and patterns in the data.

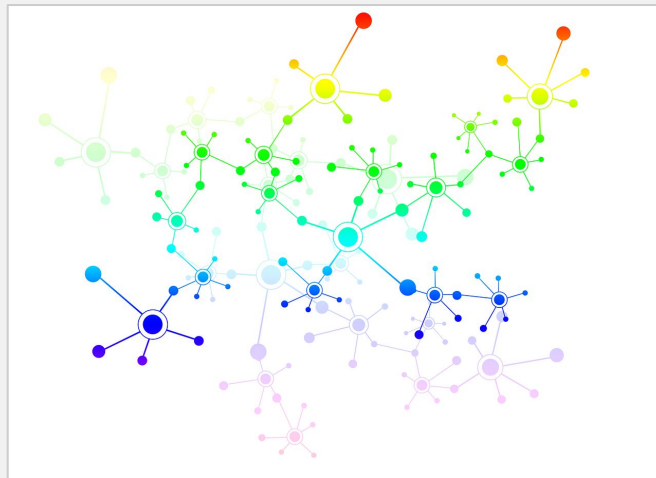
However, these models are limited by their sequential nature, which makes it difficult to connect information beyond basic word order in sentences.



Transformers vs. RNN/LSTM

Transformers are **the next evolution** after RNNs and LSTMs.

Unlike their predecessors, transformers make it possible for **all words to have direct connections with all other words**. They are built from multiple components.



An Example: Why Transformers Matter

RNNs and LSTMs often struggle to:

- ❑ Keep numbers associated with the correct region.
- ❑ Link these figures to the final global summary.
- ❑ Track words and numbers that aren't adjacent.

Transformers handle this easily, making **accurate global conclusions** possible.

Consider this report:

Q4 Revenue Report

Americas: \$2.1M (↑10%)

EMEA: \$1.8M (↓5%)

APAC: \$1.5M (↑15%)

Therefore, global performance increased by 6% overall (weighted average).

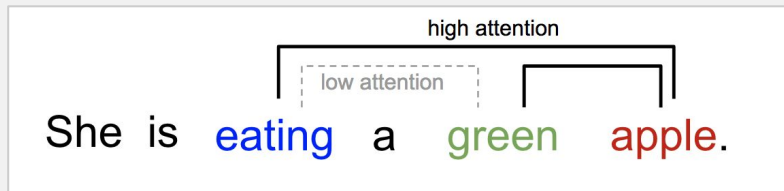


Attention Please!

Transformers make it possible for all words to have **direct connections with all other words**, which makes more complex, global conclusions possible.

Keeping track of all of those connections is very memory intensive and we don't necessarily need to focus on all of them all of the time.

Attention is the solution.



Attention Please!

Attention selectively focuses on various parts of the model inputs to create a piece of the output.

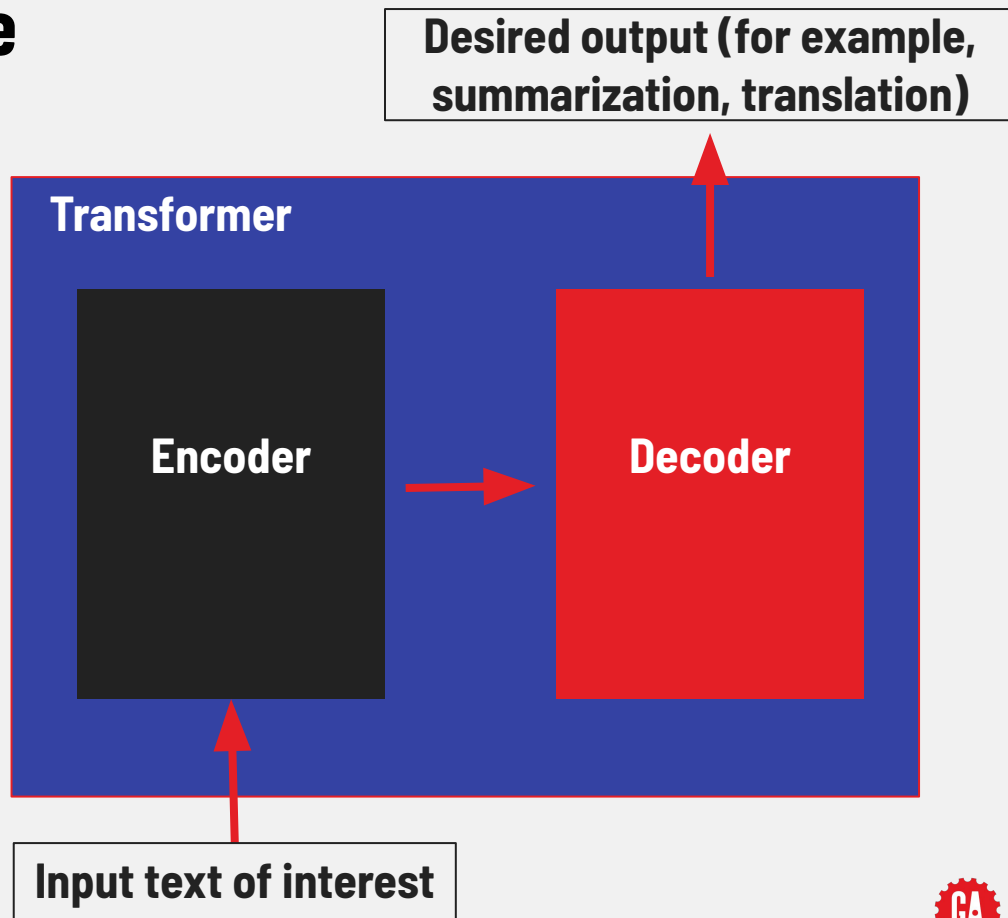
The exact mechanism of how the model uses attention depends on the model architecture.

The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .

Transformer Architecture

An **encoder** converts input into a compressed representation (encoding) capturing key features and relationships.

A **decoder** uses this encoding to generate output, step by step.



Transformer Components and Uses

Encoders and decoders can be used alone or together to accomplish NLP tasks.

Model Family	Example Models	Commonly used for
Encoder Only	BERT, RoBERTa	Sentence Classification, Named Entity Recognition (NER),
Decoder Only	GPT, Llama, DeepSeek-V3	Generative tasks, summarization, translation, Q&A.
Encoder-Decoder (Sequence-to-Sequence)	BART, T5	Generative tasks especially using multimodal data (for example images and text), time series forecasting, translation.

Encoder Models

Encoder (Only) Models

One of the most famous **encoder models** is the:

[Bidirectional Encoder Representations from Transformers \(BERT\)](#) model.

This was developed by Google in 2018.

Have you ever used Google?

Yes? Then, you've used **BERT!**

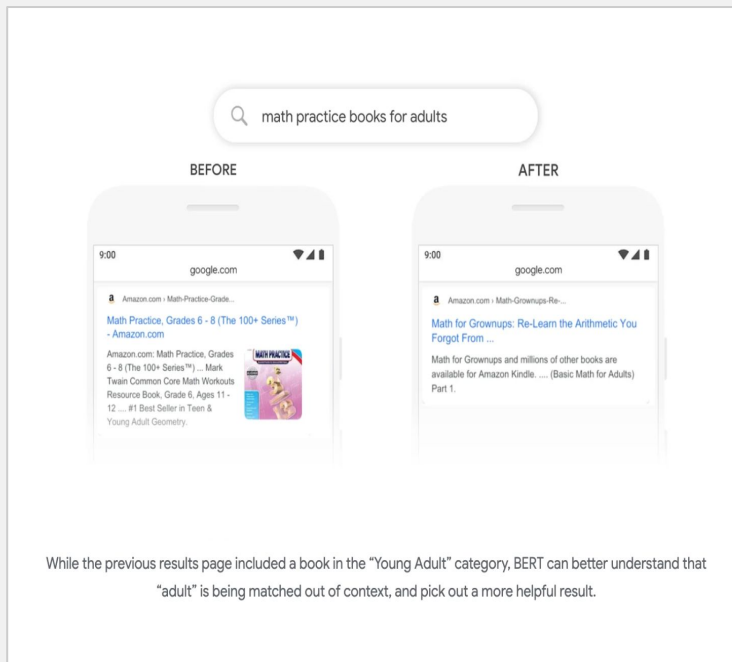


Encoder (Only) Models

Take a look at the results here.

BERT was a big advancement from simply matching keywords to being **context aware**.

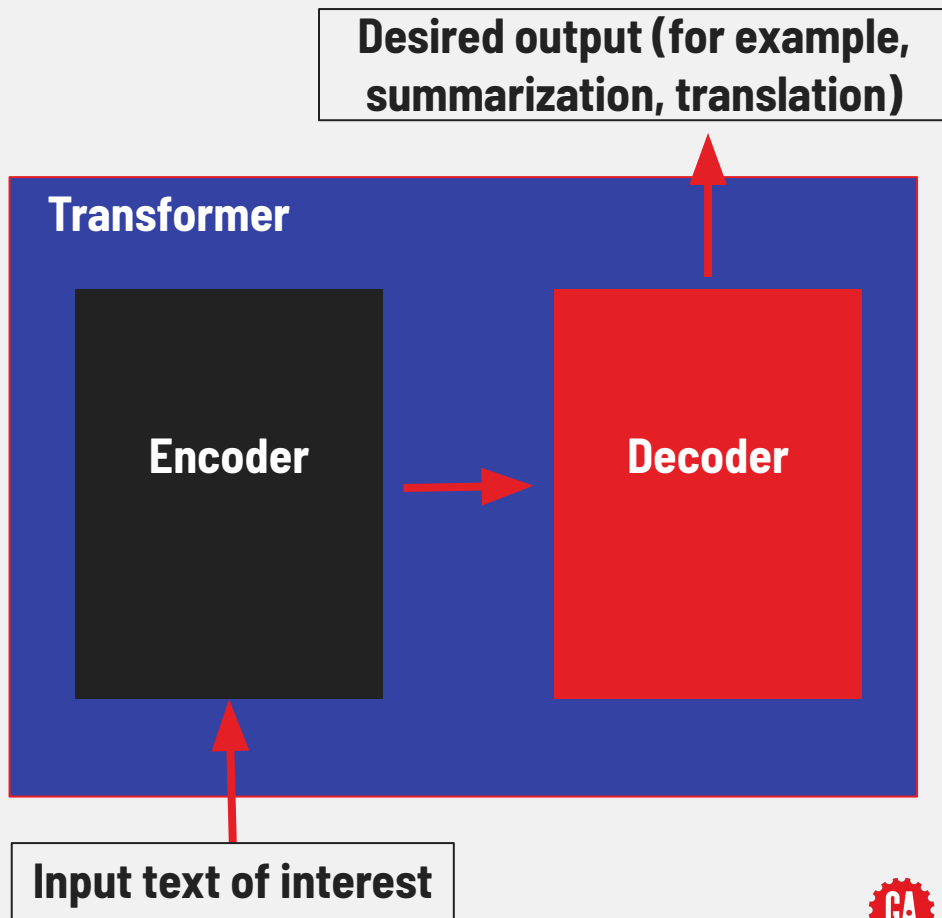
In this case it understood that “Adult” is a better match to “Grownups” than “Young Adult”.



Encoder Only: Review

Here is the **full transformer** architecture we saw earlier.

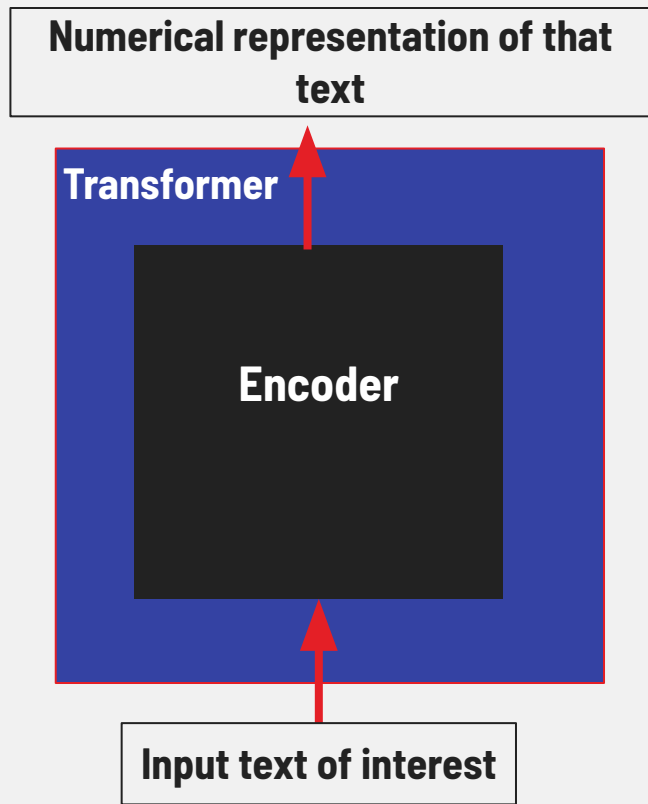
For an **encoder only architecture** we drop the decoder and focus only on the encoder side.



Encoders: How do they work?

Encoders use **bidirectional self-attention**, meaning all tokens can attend to all other tokens in either direction.

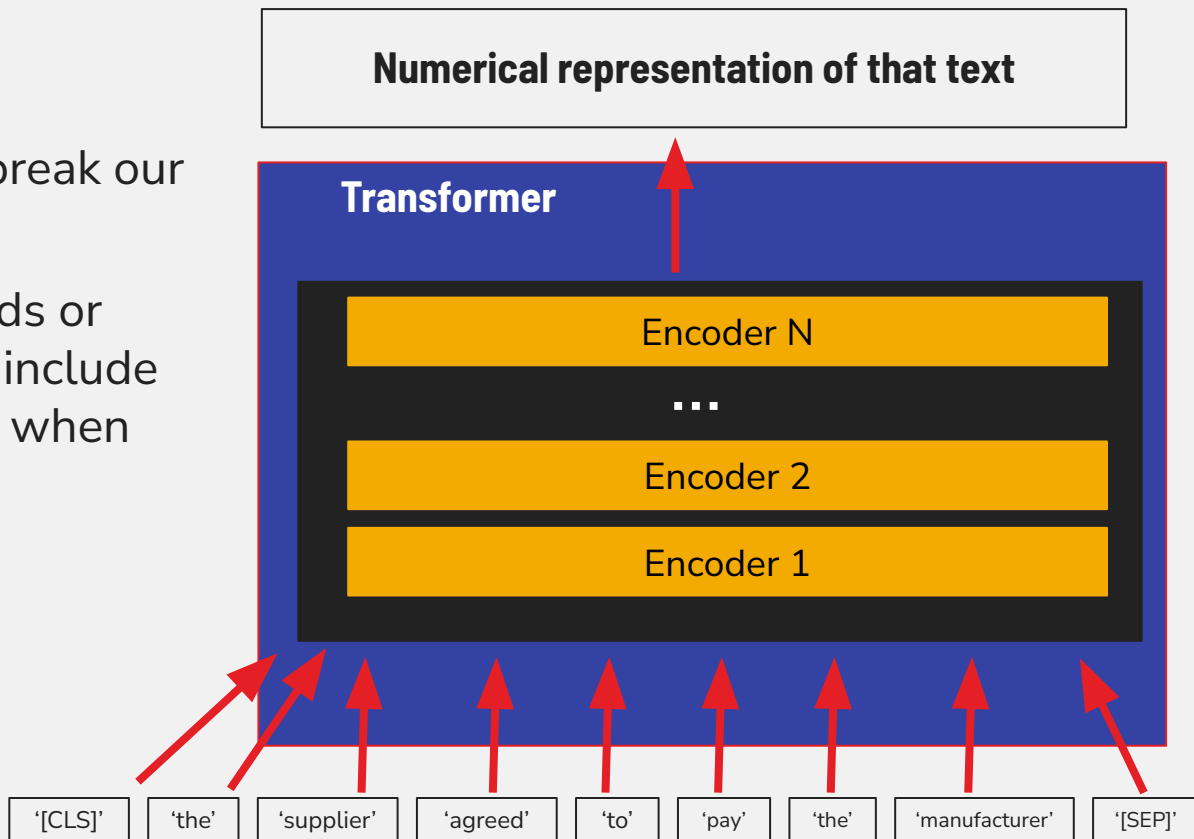
At a high level encoder models take in text and output a numerical representation of that text.



Encoders: Inputs

More specifically, we first break our text into tokens.

These could be whole words or pieces of words, and often include helper tokens that indicate when sections start and end.



Encoders: Inputs Example

Take the sentence:

The supplier agreed to pay the
manufacturer

It is broken into the following tokens
using the BERT tokenizer:

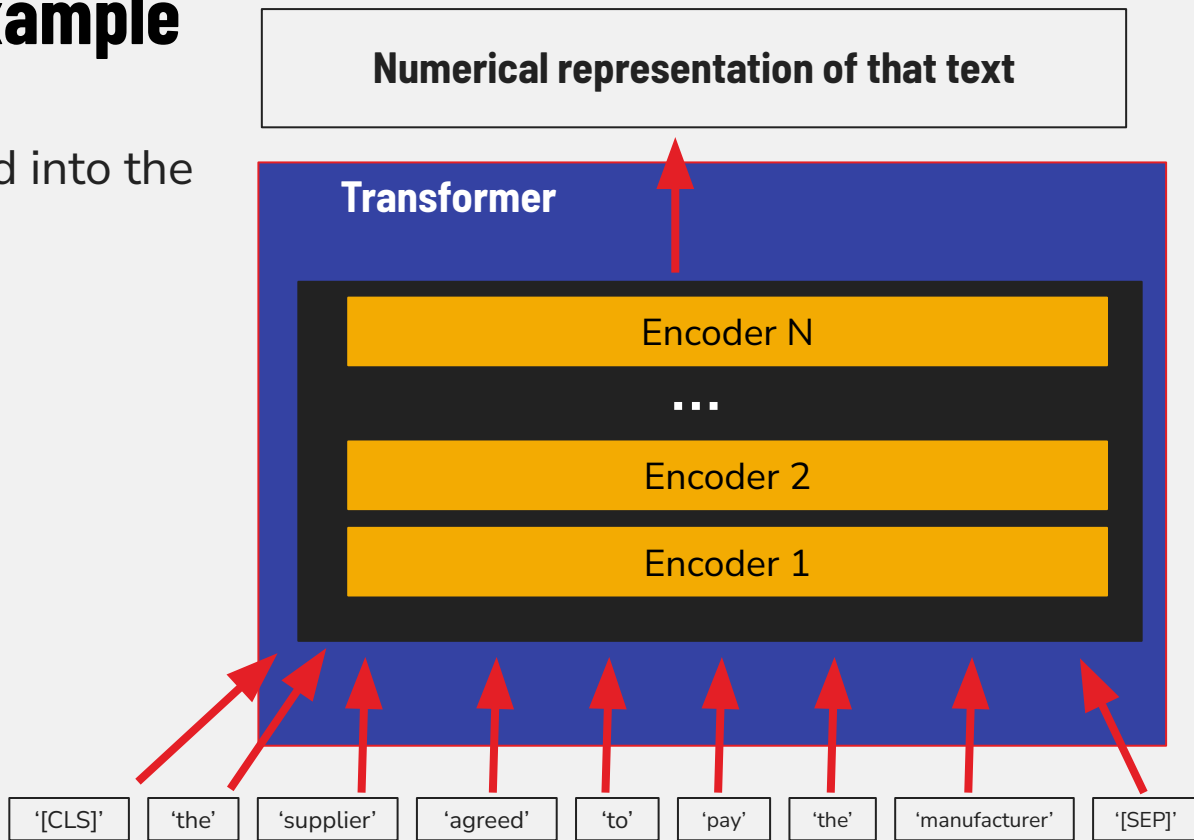
```
['[CLS]', 'the', 'supplier',  
'agreed', 'to', 'pay', 'the',  
'manufacturer', '[SEP]']
```

[CLS] indicates the start of a
passage.

[SEP] indicates the end of a
sentence or the end of the passage.

Encoders: Inputs Example

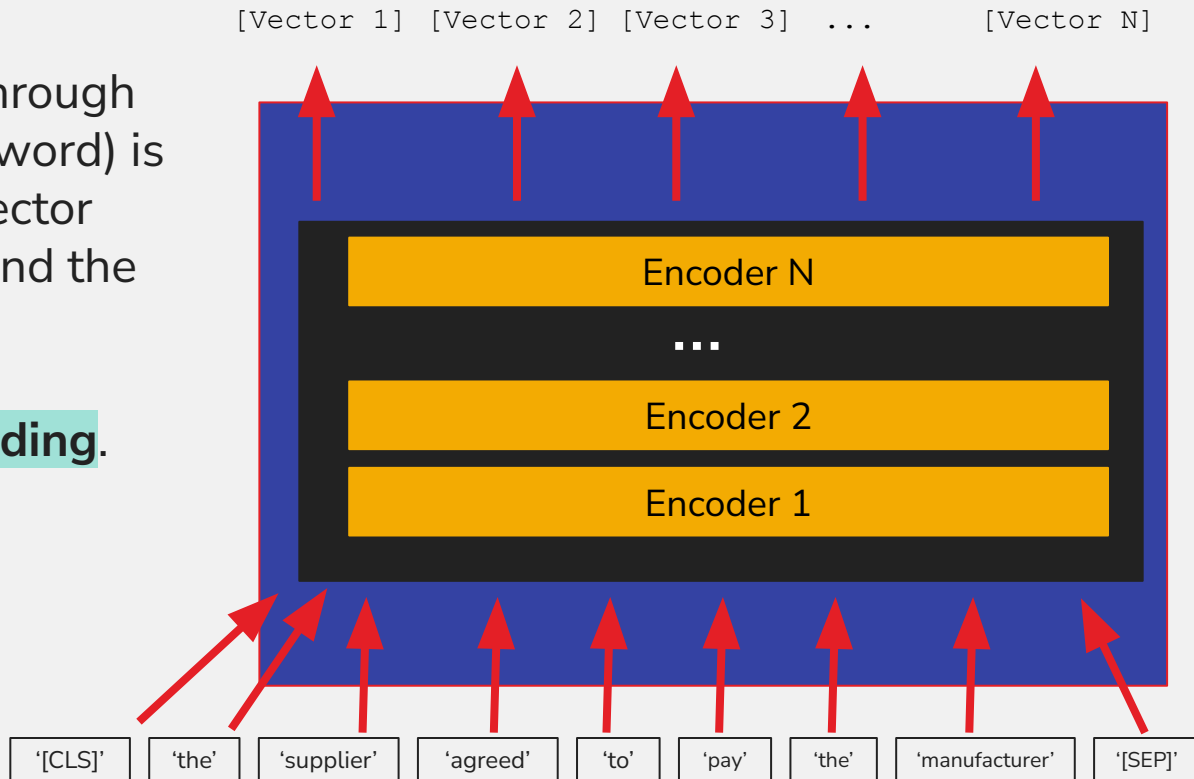
Our tokens are then passed into the encoder.



Encoders: Outputs Contextual Embeddings

As our tokens are passed through the encoder, each token(or word) is given a unique numerical vector that represents that word and the words around it.

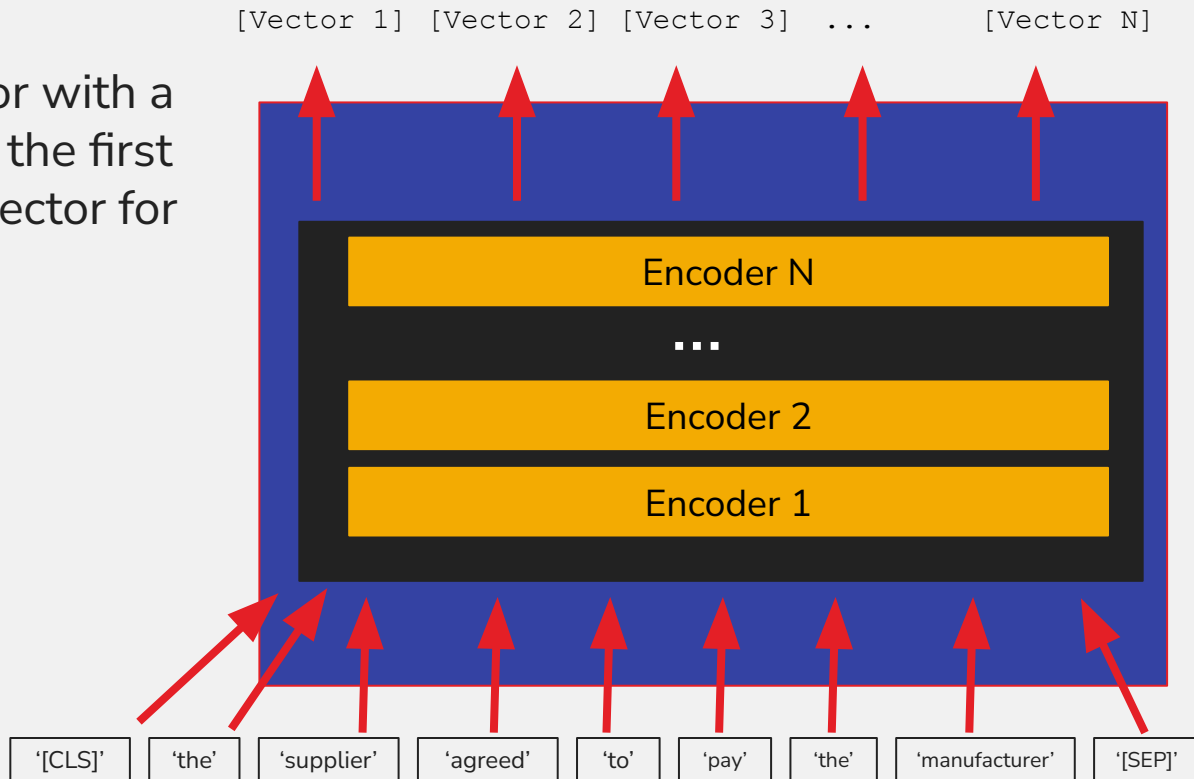
This is a **contextual embedding**.



Encoders: Outputs Contextual Embeddings

BERT models return a vector with a dimension of 768. Here are the first 10 of those values for the vector for 'pay':

```
[1.75651863e-01  
-1.36638544e-02  
3.45966190e-01  
2.97459602e-01  
1.11937486e-01  
4.56450313e-01  
-6.91465288e-02  
-9.44722354e-01  
5.65952301e-01  
-9.48831439e-06]
```



Encoders Use Cases and Examples

Sentiment Analysis

Because encoders are contextually aware they excel at sentiment analysis where word order (and negations) matter a lot.

Sentiment Example

I'm so happy today!
I'm not happy today!

Masked Language Modeling (MLM)

Where you try to predict a word given the word(s) that come before and after that word.

MLM Example

I'm ___ today!

What is NER?

Named-entity recognition (NER) is a method to identify and classify **named entities** mentioned in unstructured text into pre-defined categories.

These categories could be person names (PER), organizations (ORG), locations (LOC), geopolitical entities (GPE), vehicles (VEH), and many others.

The screenshot displays the Google Cloud Natural Language API interface. On the left, a sidebar contains links for 'Benefits', 'Demo', 'Key features', 'Customers', 'Documentation', 'Generate a solution', 'Pricing', and 'Take the next step'. The main area is titled 'Try the API' and shows a sample text analysis. The text is: 'Google, headquartered in Mountain View (1600 Amphitheatre Pkwy, Mountain View, CA 940430), unveiled the new Android phone for \$799 at the Consumer Electronic Show. Sundar Pichai said in his keynote that users love their new Android phones.' The text is annotated with entity tags: (Google)₁₀, (Mountain View)₁₁, (1600)₁₂, (1600 Amphitheatre Pkwy, Mountain View, CA)₅, (1600 Amphitheatre Pkwy)₆, (Mountain View)₁₁, (CA)₈, (940430)₃, (Android)₇, (phone)₁₄, (\$799)₁, (799)₄, (Consumer Electronic Show)₉, (Sundar Pichai)₁₂, (users)₁₆, (love)₁₅. Below the text, a table lists the identified entities with their categories and corresponding labels.

Entities	Sentiment	Moderation	Categories
1. \$799			PRICE
2. 1600			NUMBER
3. 940430			NUMBER
4. 799			NUMBER
5. 1600 Amphitheatre ...			ADDRESS
6. 1600 Amphitheatre ...			LOCATION
7. Android			CONSUMER GOOD
8. CA			LOCATION
9. Consumer Electroni...			EVENT
10. Google			ORGANIZATION
11. Mountain View			LOCATION
12. Sundar Pichai			PERSON



Encoder Model - NER

Let's see it for ourselves!

- Explore the Hugging Face Pipeline to complete Named Entity Recognition (NER)
- Use BERT based **encoder models** to complete different versions of NER in **encoders-NER.ipynb**



- ☐ What does each model output?
- ☐ How much time does it take to run each model?
- ☐ Which would you choose for your task?

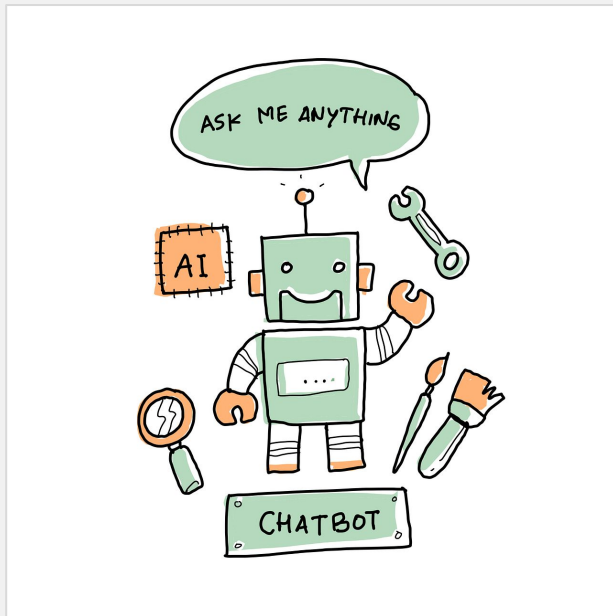
Decoder Models

Decoder (Only) Models

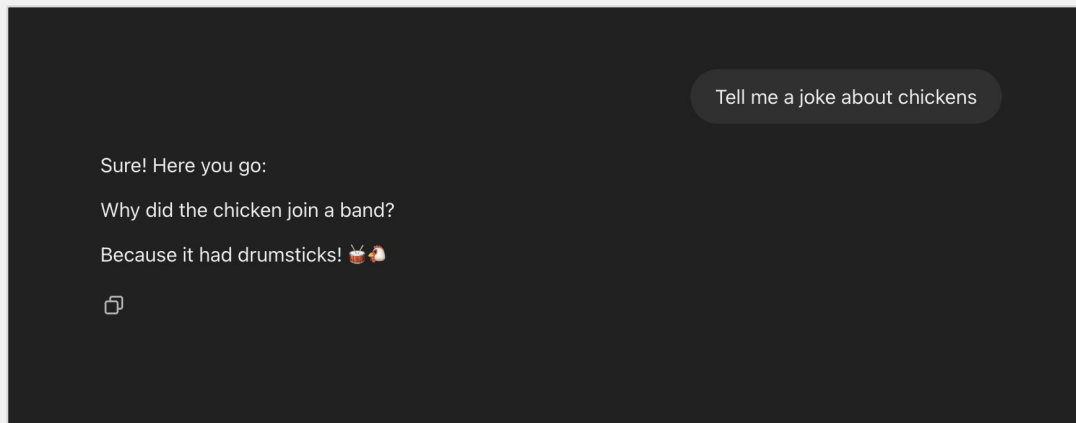
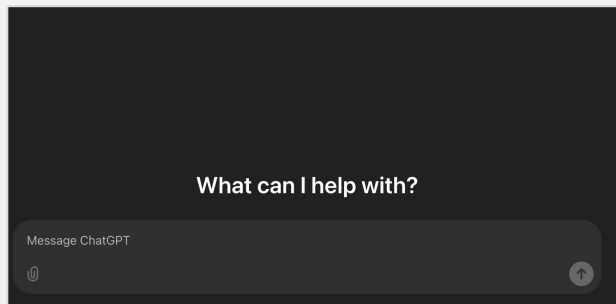
Decoder models have become the go to model for many AI companies.

ChatGPT, Claude, and Llama are all examples of a decoder only model.

Decoder models are especially good at generative tasks, such as answering questions.



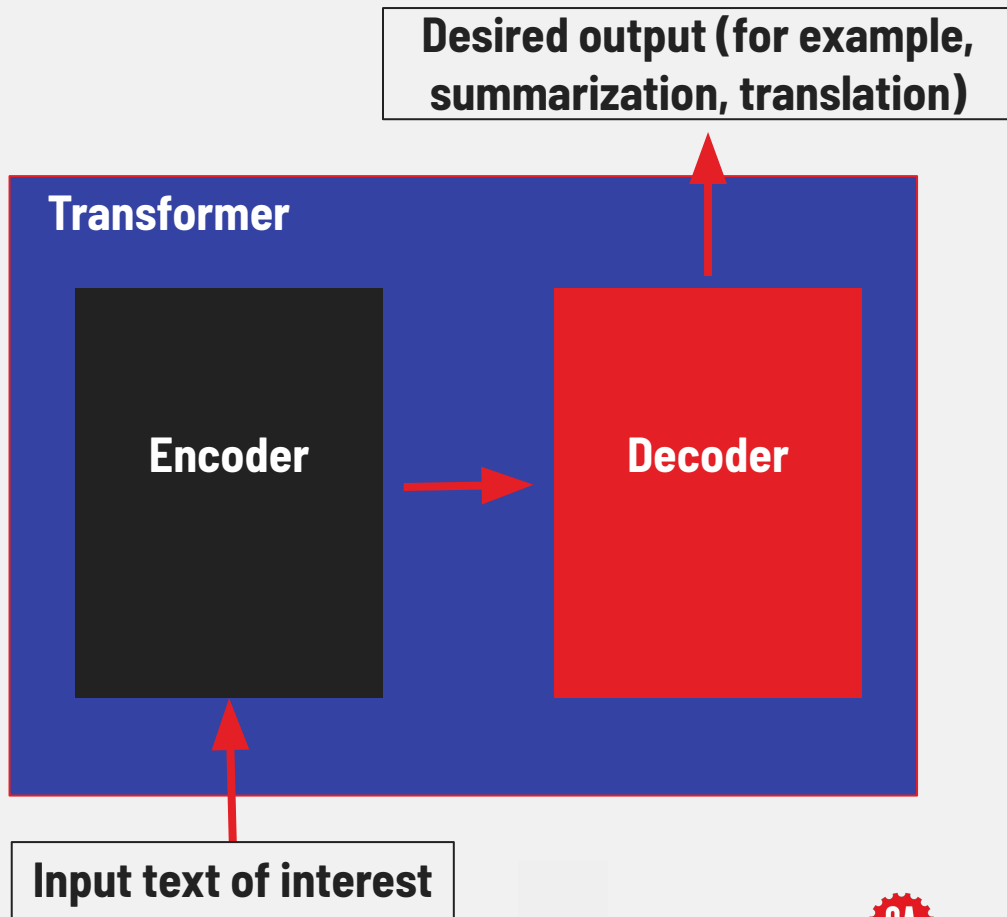
Decoder (Only) Models: An Example



Decoder Only: Review

Recall the full transformer architecture we saw earlier.

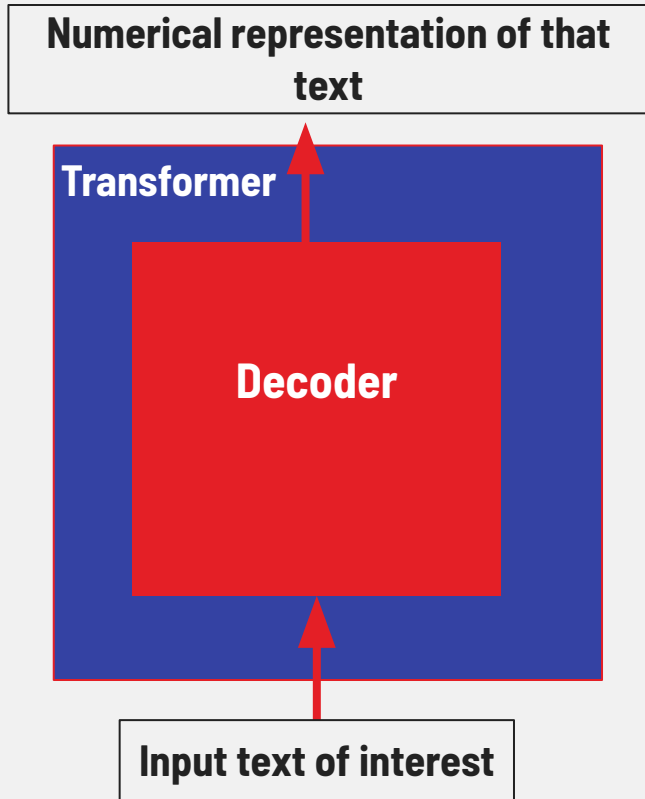
For a decoder only architecture we drop the encoder and focus only on the decoder side.



Decoders: How do they work?

At a high level, decoder models take in text and output a numerical representation of that text. This seems very similar to encoders but the difference lies in what is used as context.

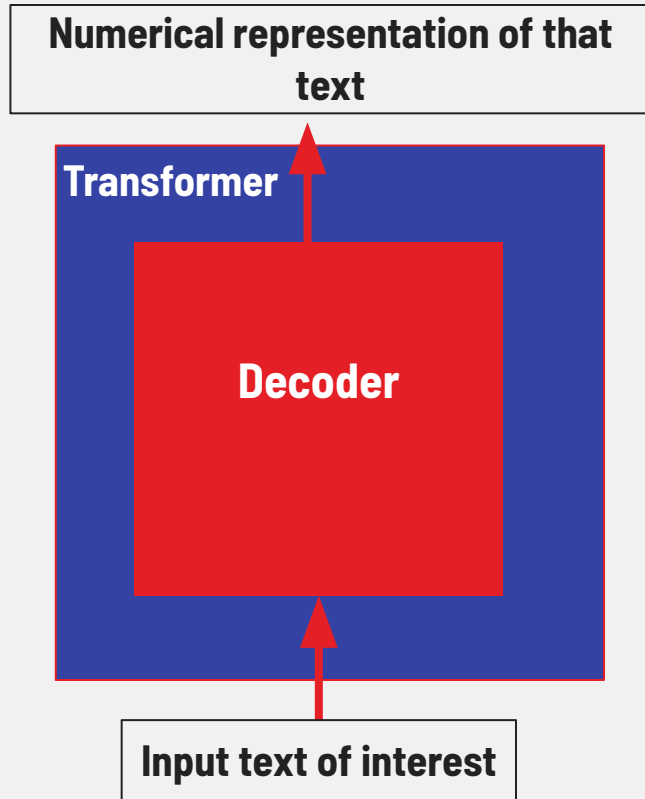
Decoders used **masked self-attention** meaning that a given token can only attend to tokens that came before (or after!) but not both directions.



Decoders: How do they work?

Decoders create the numerical representation of the word that accounts for the words from one direction (either the words before it, or those after it, but not both) as their context. This is different than the bidirectional encoder we saw before.

Let's dive into the details of how this works.



Decoders: Masked Self Attention

More specifically, we first break our text into tokens. These could be whole words or pieces of words, and often include helper tokens that indicate when sections start and end.

For example the sentence:

The supplier agreed to pay
the manufacturer

'<S>'

'the'

'supplier'

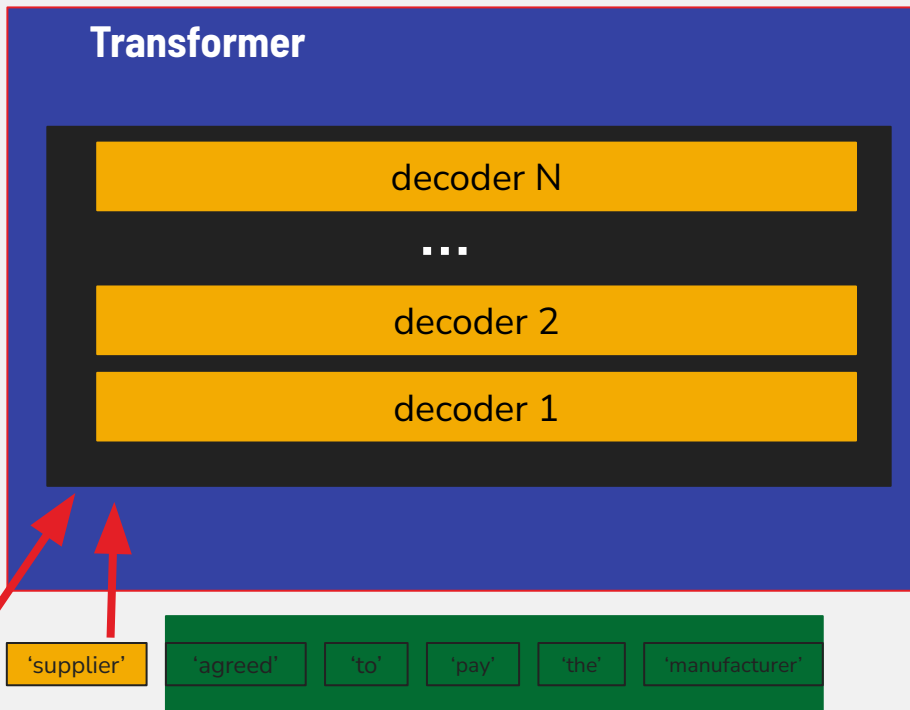
'agreed'

'to'

'pay'

'the'

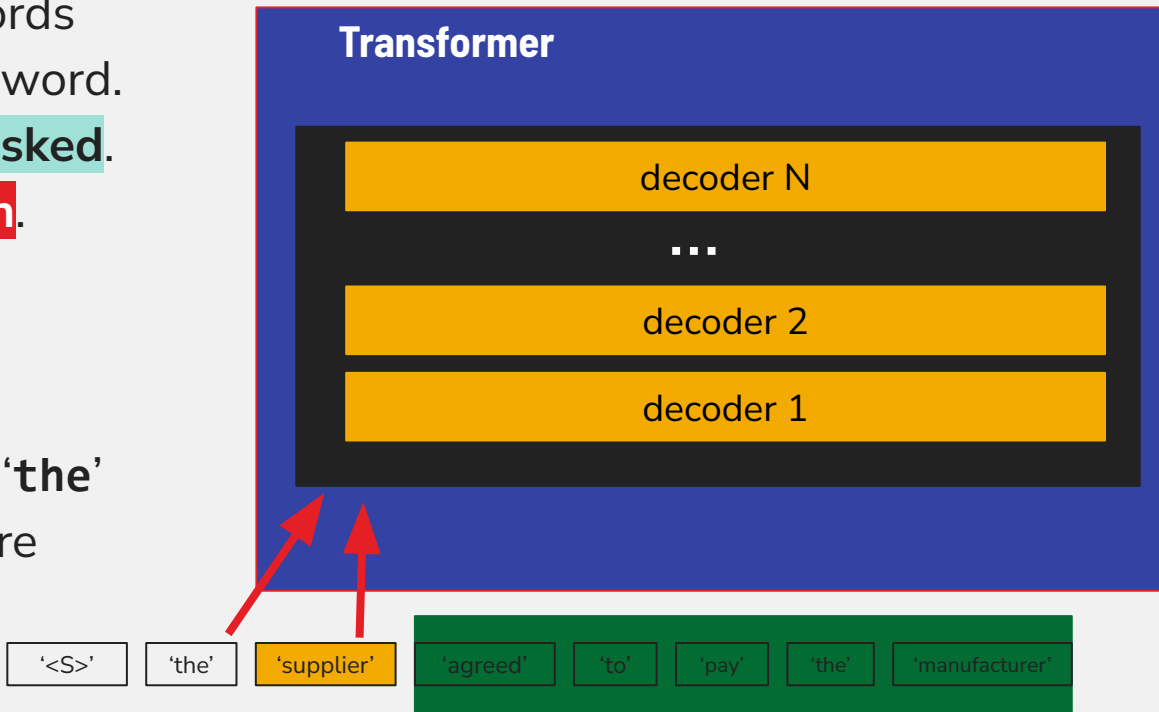
'manufacturer'



Decoders: Masked Self Attention

The decoder will only use words that come before the current word. All the **words after it are masked**. This is **masked self-attention**.

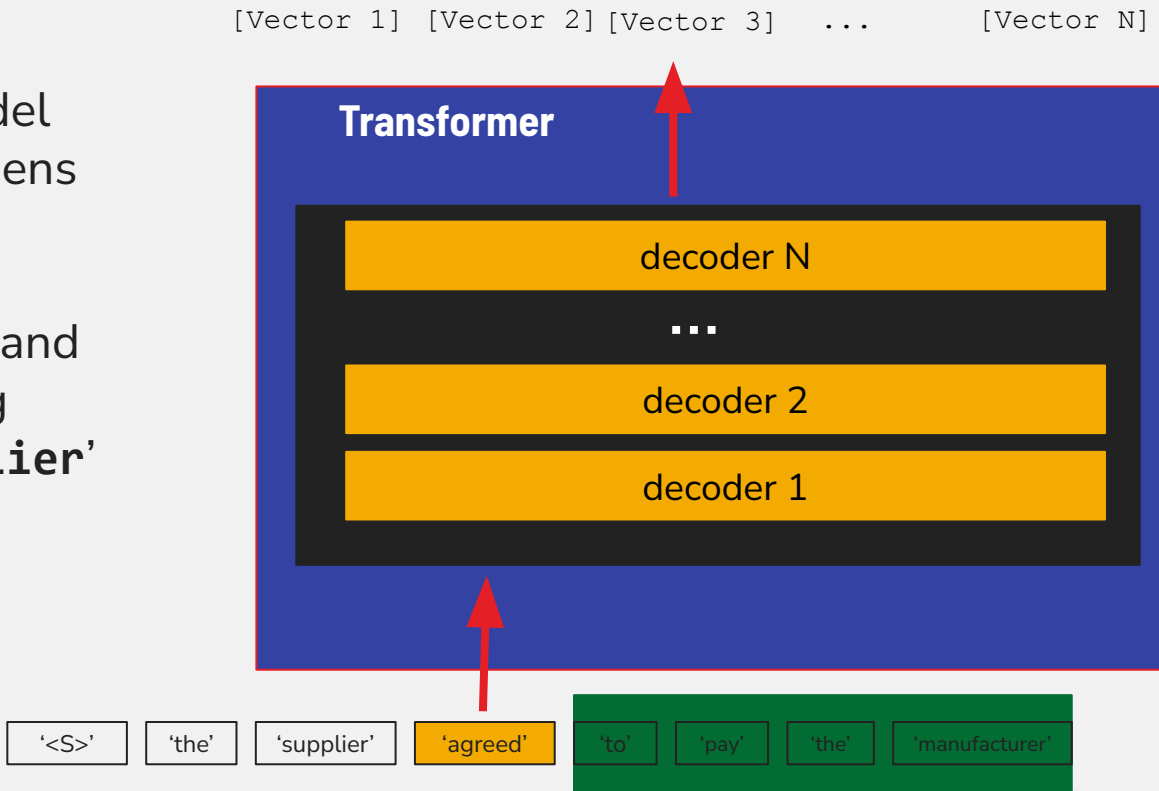
In this example, the word **'supplier'** will only take the starting token **'<S>'** here and **'the'** as context. All other words are masked (shown in green).



Decoders: Masked Self Attention

As we move on into the model process, let's see what happens for 'agreed'.

Notice the mask has shifted and now we will use the starting token '<S>', 'the', and 'supplier' to generate the vector for 'agreed'.



Decoders for Text Generation

Text Generation

Decoders are great at text generation, and are widely used by AI companies.

Generation example

Prompt: Tell me a joke about chickens

Reponse: "Sure, here you go: .."

see prior slide for the full joke :)

Decoder –Text Generation

Let's see it for ourselves!

- ☐ Explore the Hugging Face Pipeline to complete generation task with the GPT-2 decoder model.
- ☐ You will use **decodermodels** to generate product reviews.
- ☐ Open **decoders-generation.ipynb**.



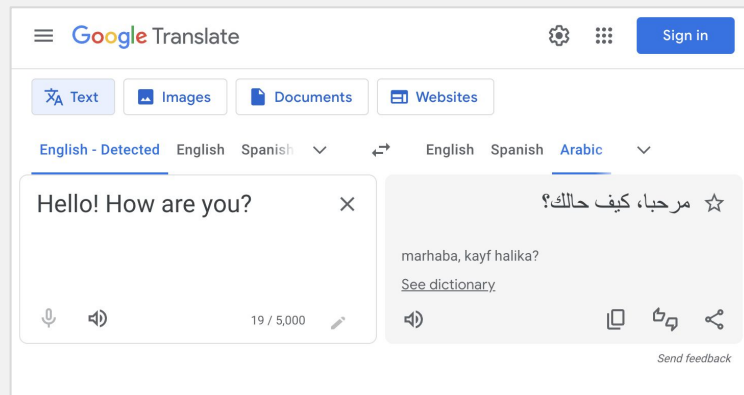
- ☐ What does each model output?
- ☐ How much time does it take to run each model?

Encoder-Decoder Models (seq2seq)

Encoder-Decoder Models

Encoder-Decoder models (also known as seq2seq or sequence-to-sequence models) are most commonly used for complex generation tasks and for new applications such as time-series forecasting and multi-modal applications.

One seq2seq application you may have used is **Google Translate**. Let's see how it works!

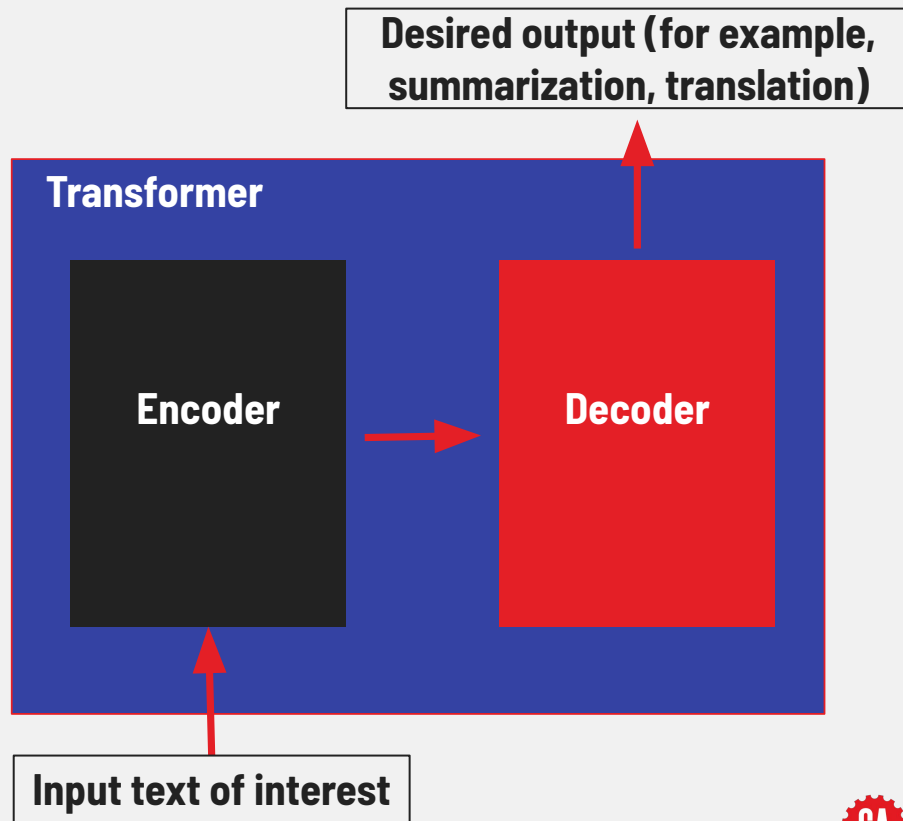


How Encoder-Decoder (seq2seq) Models Work

Recall the full transformer architecture we saw earlier. We will now use both parts!

Seq2Seq models use 3 types of attention: **bidirectional self-attention** (encoder), **masked self-attention** (decoder), and **cross-attention**, which attends to the encoder tokens.

Let's see it in action.



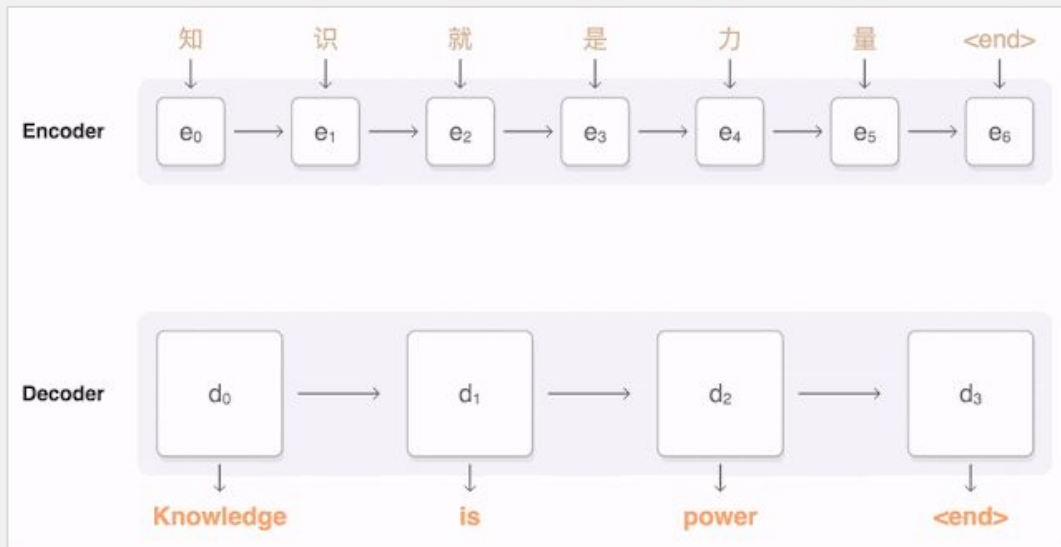
Encoder-Decoder Models

Let's dive into more detail using an example from **Google Translate**.

Input: Chinese sentence

Step 1: Encoder

Chinese words are input into the **encoder** generating a vector of all words read in so far. Once it reaches the **<end>** we move to step 2.



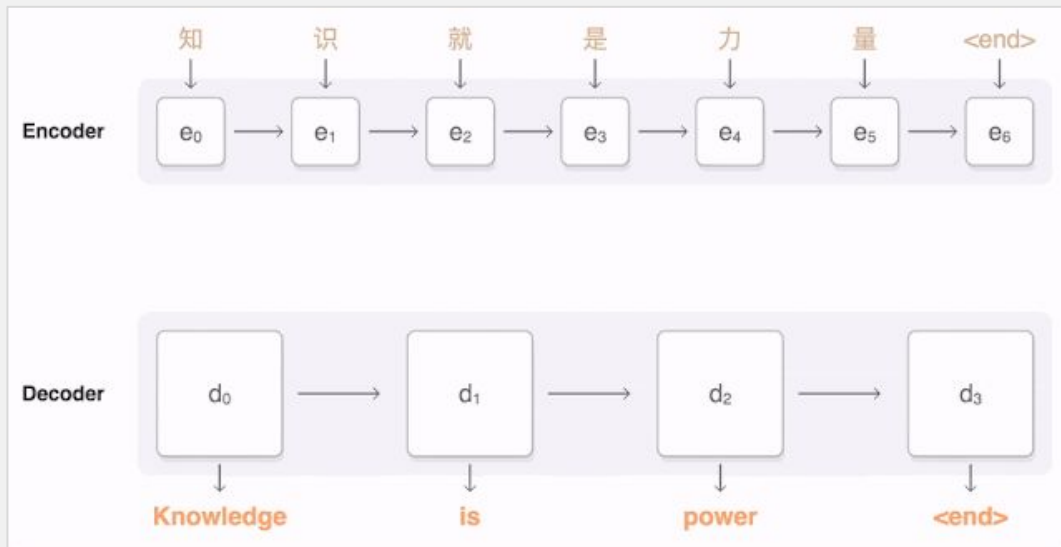
Encoder-Decoder Models (cont.)

Let's dive into more detail using an example from **Google Translate**.

Step 2: Decoder

The **decoder** then starts using a weighted distribution of the vectors from the encoder (purple lines indicate weight) to generate the translated sentence.

Output: English sentence



Encoder-Decoder Use Cases

Translation

This is a classic example from Google Translate

Q&A

The T5 algorithm is especially good at Q&A systems

Time Series Forecasting

A growing area of research particularly in energy applications

Multi-modal Applications

These models may be particularly powerful for moving between different data types (for example, images and text)

Encoder-Decoder – Translation Summarization

Let's see it for ourselves!

- ☐ Explore the Hugging Face Pipeline to complete a translation and summarization task.
- ☐ You will use **encoder-decoder (seq2seq) models** to translate and summarize documents **seq2seq-translation-summarization.ipynb**

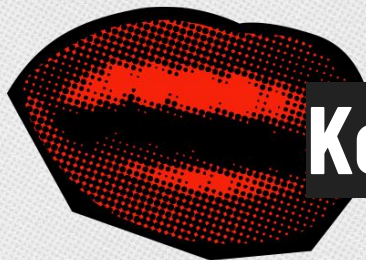


Using Transformers

How could you apply transformers to your work?

- ☐ Define an NLP (or multi-modal!) task.
 - ☐ Discuss if any of the transformer architectures would be useful.
 - ☐ Determine which type of model would best meet your needs.
 - ☐ Look up the model on Hugging Face and read the model card.
- ☐ Don't forget to consider cost of compute
 - ☐ Is there a simpler way to achieve your goal?

Bring It Home



Key takeaways



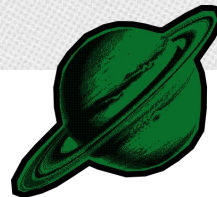
**Transformers are
extremely useful!**

You have been using them
for years!



You have options!

Select the transformer
architecture that best
meets your needs.



**Don't forget to think
about compute time
and cost**

Generally, the more
complex the model the
more expensive it is to run.

DON'T FORGET:

Exit tickets



